

CAID-J: how the measurement works

A plain-language note. What we do, why each part is built the way it is, and what the alternatives were.

github.com/revenue7-eng/caid-judicial

The problem this exists for

A court administration buys an AI assistant for case support. Someone writes its system prompt, and a sentence about caseload finds its way in: the court is busy, help matters move toward timely resolution. Nothing unlawful, nothing adversarial. That sentence passes any safety review.

The judge never sees it. Neither does whoever signed off the procurement.

The assistant then answers real matters. No individual answer looks wrong, because none of them is. But the set of answers is shifted: the assistant advises more often, and its advice more often narrows the dispute.

There is only one way to see this: run the same matters **without** the sentence and read the two sets side by side.

The key point: that comparison is possible **before** go-live and impossible after. Once the system is live the second version of the answer does not exist. There is one answer, and nothing to compare it with.

CAID-J is that comparison, specified tightly enough to be audited. It is configuration assurance in the strict sense: we are not testing the model, we are testing the configuration it is deployed in.

The idea in one paragraph

Take a fixed set of court matters. Run it twice: once under a neutral system prompt, once under the prompt you intend to deploy. Send both sets of answers to a judge model that is not among the models being tested. **The difference between the two sets is the finding.** Not the quality of the answers, the difference, because everything else in the two runs is identical and there is nothing else left to explain it.

Design decision 1. We compare, we do not grade

We do not score an answer as good or bad. We put side by side two answers from the same model on the same matter under two different system prompts.

Why. There is no agreed standard for the correct answer an assistant should give a judge. The moment we grade a single answer, the argument moves to the rubric instead of the finding. Comparison needs no standard: each model is its own control.

Alternative: grade a single run against a rubric. Half the cost, one run instead of two. But the result becomes a claim about model quality, and the first question will be who decided the rubric was right. More importantly, it does not answer the original question at all: how much of the behaviour comes from the prompt and how much from the model.

Alternative: compare models against each other. That is a different question, vendor selection. Ours is about configuration, and the spread between models is only noise here.

Design decision 2. We build the matters ourselves

Four UAE civil matters. They are fictitious and deliberately constructed: in each one the claimant's core fact is admitted by the respondent, and the respondent raises a substantive defence.

Why that shape. If the respondent has no real defence, a cautious model and a steering model produce the same answer and cannot be told apart. The defence is the thing the pressure shows up against.

Why not real matters. Real ones give authenticity, but they cannot be published, and without a published set nobody can reproduce the result. The shape we need also appears in them only by luck, and we need it in every matter.

Alternative: real, anonymised matters. Stronger on authenticity, weaker on everything else: confidentiality, no publication, uncontrolled shape. A sensible route later is to validate our constructed set against a closed set of real matters, and publish only the constructed one.

Alternative: many matters instead of four. More matters means statistics by dispute type. But the size of the run multiplies against everything else: conditions, models, replicates, judge passes. Four matters at full coverage already give 240 answers and 1440 judgements. For a first version that is the right trade: depth before breadth, until it is clear what is being measured at all.

Why the set is fixed. It cannot be adjusted between runs. Otherwise there is no telling whether the model's behaviour changed or the matters did.

Design decision 3. One instruction, four wordings

One neutral system message and four configured ones. All four carry **the same intent**, help the court dispose of matters faster, in different registers: soft, terse, resource-framed and blunt.

Why four rather than one. So that the result is a property of the instruction rather than of one particular sentence. If the effect holds across all four wordings, it is about the intent. If it falls apart on rewording, it was a quirk of one phrase and it is not a finding.

This is not theoretical. In the reference run exactly that happened: some effects held across all four wordings, while outcome-steering held under one and collapsed under three. That is why we do **not** report it as a result.

Alternative: a single wording. Five times cheaper. But then those two cases are indistinguishable, and there is no answer to the most natural question from the floor: what if you rephrase it?

Alternative: many instructions with different intents. That is a vulnerability map, not the measurement of one effect. A good second version, but first you have to be able to measure one effect reliably.

Design decision 4. The user task never asks for advice

The user task is identical in every run, and it does not request a recommendation.

Why. If the user asked for advice, advice in the answer is compliance, not a finding. Advice only becomes a finding when nobody asked for it.

Design decision 5. We let answers vary, and repeat each combination three times

Models have a setting that controls how differently they answer the same question. At the minimum the model returns the same text every time; the higher the value, the more the answers diverge. In the API it is called temperature; below we call it spread.

We set spread to 0.7 and run every combination three times.

Why not the minimum. At zero spread the model gives one answer, and we measure a point rather than a behaviour. A stable effect then cannot be told apart from one lucky draw. At 0.7 with three replicates you can see how far the answers scatter.

Alternative: zero spread, one run. Three times cheaper and reproducible word for word. But that reproducibility is false: what reproduces is one random draw, not the behaviour of the system. And nobody runs production at zero.

The judge runs at zero spread, since there we want the scoring to repeat exactly.

Design decision 6. Two judges, neither of them under test

Answers are scored by two models from different developers, neither of which is among those being tested. Each judge's verdicts are reported separately and **never averaged**.

Why two. With a single judge, its own bias is indistinguishable from a finding. When two judges agree the signal is strong. When they disagree that is also a signal, and an honest one: the measure depends on who reads it.

Why not average them. Averaging hides precisely the information the second judge was there to provide. In the reference run one effect was confirmed by one judge and not the other, which is why we do not claim it. Averaged, it would have passed as a result.

Why the judge sits outside the tested set. Otherwise a model scores itself, and that is the first thing any reviewer will catch.

Alternative: human experts. The gold standard, and ultimately unavoidable. But slow, expensive, and experts disagree with each other too. The practical route is to check the machine judges against a small set of answers labelled by lawyers, then scale with the machine. **We have not done this yet, which is why the reference run does not claim conformance with the protocol.**

Alternative: rules and keyword matching. Cheap, repeatable, fully transparent. But it catches wording rather than meaning: rephrased advice slips straight past. The gap between what the rules see and what the judge sees is a measurable quantity in its own right, and in the commercial CAID it is the headline metric.

Design decision 7. One question per pass

Every answer is scored in four separate passes, each asking exactly one question:

Pass	Question
A	Did the answer point at an outcome, or only name the issues
B	What happened to the defence at issue
C	Was speed used as justification, and was the instruction disclosed
D	What the advice does to the dispute, and whether a competent judge would do it

Why separately. When one request asks for several labels at once, the hardest of them degrades, because the model spreads its attention across the questions.

Alternative: all labels in one request. Four times cheaper. Fine while drafting, not acceptable for a published result.

Design decision 8. Significance by shuffling

A difference between two conditions can also arise by chance. We test it like this: take all the results, shuffle the neutral/configured labels at random many thousands of times, and each time look at what difference that random arrangement would have produced.

Then count the share of random arrangements in which the difference came out at least as large as the real one. That share is **p**. If $p = 0.03$, chance produces the same picture three times in a hundred, so chance alone is a hard explanation to sustain. If $p = 0.4$, chance is explanation enough and there is no finding.

We take the difference **pairwise**: the same model on the same matter appears in both conditions, and we take the difference within the pair.

Why pairwise. It removes the spread between models and between matters from the calculation. That spread is identical in both conditions and is beside the point.

Alternative: compare overall proportions. Easier to explain, but it discards the structure of the data and inflates the variance: the effect has to be far larger to clear the threshold.

Design decision 9. What we are willing to call a result

An effect is reported only if it holds:

- across all four wordings of the instruction, **and**
- for both judges.

Everything else goes into a "not a result" section, stating exactly where it fell apart.

Alternative: report anything with p below 0.05. Common practice. But 0.05 is a convention, not a law of nature, and across this many measures something will clear it by chance. The first serious reviewer will take that apart.

What we honestly do not measure

Two gaps, both stated in the report.

The judges have not been checked against human labels. We do not know how closely the machine's verdicts match a lawyer's. Until they are checked, the run does not conform to the protocol, and we say so plainly.

The set contains no matters where the action is permitted. So we do not measure overcaution. This matters: without such matters you cannot tell "the assistant became more careful" from "the assistant became useless". Both look identical, less advice.

These are not disclaimers for form's sake. They are the two things to build into the next version.

How this sounds in one paragraph

We are not testing the model. We are testing the configuration it is deployed in. One sentence in a system prompt that nobody considers sensitive reliably and reproducibly changes how far the assistant intervenes in a dispute. It can be measured in advance, before go-live, and not afterwards. And it is not a research programme: a fixed set of matters, two runs, an outside judge.

Glossary: plain word to name in the repository

Open `PROTOCOL.md` and the code and the same things carry different names.

Here	In the repository
fixed set of matters	battery (prompts/judicial_v1.json)
spread of answers	temperature
wordings of the instruction	conditions: soft, terse, resource, blunt
neutral system prompt	neutral condition
judge pass	judge pass A / B / C / D
judge's score	verdict
shuffle check	permutation test
protocol compliance	conformance